

Algoritmos e Programação I

Prof. Dr. Amaury Antônio de
Castro Junior

Módulo 3 - Variáveis compostas: *strings*, vetores e matrizes

Unidade 2 - Vetores e Matrizes

O que já estudamos?

- Tipos simples (números, caracteres) e comandos básicos de manipulação de dados.
- Estruturas de controle de fluxo para tomada de decisão (*if/else*).
- Estruturas de controle de fluxo para repetição (*for* e *while*).
- Definição e manipulação básica de *strings*.

Por que usamos Vetores e Matrizes?

- Variáveis simples armazenam apenas um valor.
- Muitos problemas reais precisam guardar conjuntos de valores relacionados.
- Exemplo: notas de alunos, tempos de corrida, temperaturas, coordenadas.

Vetores como listas em Python

- Conceito de vetor:
 - Variável composta unidimensional.
- Armazena vários valores em uma única variável (identificador).
- Cada valor é acessado individualmente por um índice (posição no conjunto).
- Em Python, vetores são representados por listas.

Vetores como listas em Python

- Para acessar (ler ou escrever) um elemento em uma posição do vetor, basta informar a posição entre colchetes:

```
notas = [8.0, 5.5, 7.0]
```

- `notas[0]` → 8.0
- `notas[1]` → 5.5
- `notas[2]` → 7.0

- O primeiro índice é 0.
- O último índice é `len(vetor) - 1`.
- Acesso direto: `vetor[indice]`.

Percorrendo os elementos de um vetor

- Podemos usar o comando **for**.
- Permite acessar todos os elementos do vetor.

```
notas = [8.0, 5.5, 7.0]
```

- `notas[0]` → 8.0
- `notas[1]` → 5.5
- `notas[2]` → 7.0

```
for i in range(len(notas)) :  
    print(notas[i])
```

Percorrendo os elementos de um vetor

- Podemos usar o comando **for**.
- Permite acessar todos os elementos do vetor.

```
notas = [8.0, 5.5, 7.0]
```

- `notas[0]` → 8.0
- `notas[1]` → 5.5
- `notas[2]` → 7.0

```
for i in range(len(notas)) :  
    print(notas[i])
```

Percorrendo os elementos de um vetor

- Podemos usar o comando **for**.
- Permite acessar todos os elementos do vetor.

```
notas = [8.0, 5.5, 7.0]
```

- `notas[0]` → 8.0
- `notas[1]` → 5.5
- `notas[2]` → 7.0

```
for i in range(3):  
    print(notas[i])
```

Percorrendo os elementos de um vetor

- Podemos usar o comando **for**.
- Permite acessar todos os elementos do vetor.

```
notas = [8.0, 5.5, 7.0]
```

- `notas[0]` → 8.0
- `notas[1]` → 5.5
- `notas[2]` → 7.0

```
for i in range(3):  
    print(notas[i])
```

```
i = 0 - notas[0] - 8.0  
i = 1 - notas[1] - 5.5  
i = 2 - notas[2] - 7.0
```

Preenchendo um vetor por leitura

```
notas = []  
notas.append(float(input('Digite a nota do primeiro aluno: ')))  
notas.append(float(input('Digite a nota do segundo aluno: ')))  
notas.append(float(input('Digite a nota do terceiro aluno: ')))  
print(notas)
```

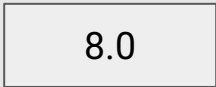
Preenchendo um vetor por leitura

```
notas = [] # o vetor deve ser definido antes de ser usado
notas.append(float(input('Digite a nota do primeiro aluno: ')))
notas.append(float(input('Digite a nota do segundo aluno: ')))
notas.append(float(input('Digite a nota do terceiro aluno: ')))
print(notas)
```

Notas → ??

Preenchendo um vetor por leitura

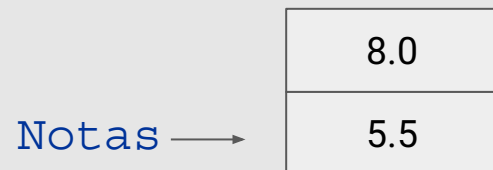
```
notas = [] # o vetor deve ser definido antes de ser usado
notas.append(float(input('Digite a nota do primeiro aluno: ')))
notas.append(float(input('Digite a nota do segundo aluno: ')))
notas.append(float(input('Digite a nota do terceiro aluno: ')))
print(notas)
```

Notas → 

- A função `append()` adiciona valores ao final do vetor

Preenchendo um vetor por leitura

```
notas = [] # o vetor deve ser definido antes de ser usado
notas.append(float(input('Digite a nota do primeiro aluno: ')))
notas.append(float(input('Digite a nota do segundo aluno: ')))
notas.append(float(input('Digite a nota do terceiro aluno: ')))
print(notas)
```



- A função `append()` adiciona valores ao final do vetor

Preenchendo um vetor por leitura

```
notas = [] # o vetor deve ser definido antes de ser usado
notas.append(float(input('Digite a nota do primeiro aluno: ')))
notas.append(float(input('Digite a nota do segundo aluno: ')))
notas.append(float(input('Digite a nota do terceiro aluno: ')))
print(notas)
```

Notas →

8.0
5.5
7.0

- A função `append()` adiciona valores ao final do vetor

Preenchendo um vetor por leitura

- E se a quantidade de valores a ser lida for muito grande?
- A leitura de valores pode ser feita por um laço para economizar linhas de código:

```
notas = []  
for i in range(40):  
    notas.append(float(input('Digite a nota do aluno {(i+1)}: ')))
```

Preenchendo um vetor por leitura

- E se a quantidade de valores a serem lidos for muito grande?
- A leitura de valores pode ser feita por um laço para economizar linhas de código:

```
notas = []  
for i in range(40):  
    notas.append(float(input('Digite a nota do aluno {(i+1)}: ')))
```

```
i = 0 - Digite a nota do aluno 1:  
i = 1 - Digite a nota do aluno 2:  
i = 2 - Digite a nota do aluno 3:  
i = 3 - Digite a nota do aluno 4:  
...
```

Operações comuns sobre vetores

Descrição da operação	Exemplo de uso
Operador <code>len</code>: retorna o tamanho (número de elementos) do vetor	<pre>>> valores = [3,1,6,7,10,22,4] >> len(valores) >> 7</pre>
Operador <code>in</code>: verificar se um elemento pertence ao vetor (teste de pertinência) e retorna o valor lógico correspondente.	<pre>>> valores = [3,1,6,7,10,22,4] >> 8 in valores >> False</pre>

Exercício resolvido 01

- Imprime **True** caso o valor "7" pertença à lista e **False** caso contrário :

```
lista = [1, 2, 3, 4]
valor = 7
resultado = False
for i in range(len(lista)) :
    if lista[i] == valor:
        resultado = True
print(resultado)
```

Exercício resolvido 02

- Faça um programa que preencha por leitura um vetor de 5 posições, e informe a posição em que um valor x (lido do teclado) está no vetor. Caso o valor x não seja encontrado, o programa deve imprimir o valor -1.

Exercício resolvido 02

```
vetor = [] # define o vetor
```

Exercício resolvido 02

```
vetor = []  
for i in range(5): # faz a leitura dos cinco elementos  
    vetor.append(int(input()))
```

```
i = 0 - lê o valor da posição vetor[0] = 3  
i = 1 - lê o valor da posição vetor[1] = 8  
i = 2 - lê o valor da posição vetor[2] = 9  
i = 3 - lê o valor da posição vetor[3] = 1  
i = 4 - lê o valor da posição vetor[4] = 4
```

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input()) # faz a leitura do valor a ser procurado
```

```
vetor = [3, 8, 9, 1, 4]
```

```
lê o valor de x = 9
```

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1
```

posicao

-1

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break
```

posicao	-1
x	9
i	0

```
vetor = [3, 8, 9, 1, 4]  
          ↑  
          i
```

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break
```

posicao	-1
x	9
i	0

```
vetor = [3, 8, 9, 1, 4]  
          ↑  
          i
```

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break
```

posicao	-1
x	9
i	1

```
vetor = [3, 8, 9, 1, 4]  
          ↑  
          i
```

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break
```

posicao	-1
x	9
i	1

```
vetor = [3, 8, 9, 1, 4]  
          ↑  
          i
```

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break
```

posicao	-1
x	9
i	2

```
vetor = [3, 8, 9, 1, 4]  
          ↑  
          i
```

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break
```

posicao	-1
x	9
i	2

```
vetor = [3, 8, 9, 1, 4]  
          ↑  
          i
```

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break
```

posicao	2
x	9
i	2

```
vetor = [3, 8, 9, 1, 4]  
          ↑  
          i
```

Exercício resolvido 02

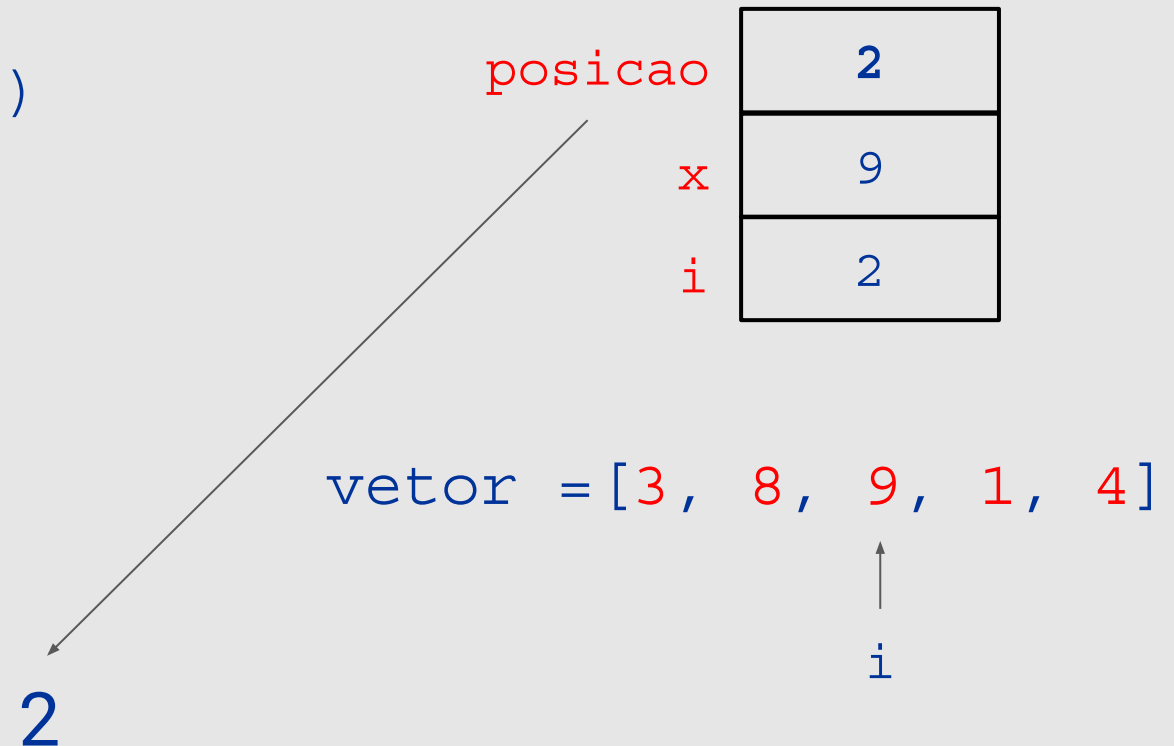
```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break
```

posicao	2
x	9
i	2

```
vetor = [3, 8, 9, 1, 4]  
          ↑  
          i
```

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break  
print(posicao)
```



Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break  
print(posicao)
```

posicao	2
x	9
i	2

vetor = [3, 8, 9, 1, 4]

E se não encontrasse o valor procurado no vetor?

Exercício resolvido 02

```
vetor = []  
for i in range(5):  
    vetor.append(int(input()))  
x = int(input())  
posicao = -1  
for i in range(5):  
    if vetor[i] == x:  
        posicao = i  
        break  
print(posicao)
```

Depois de percorrer o vetor inteiro sem encontrar a posição, o valor permanecerá -1 (que era o valor inicial).

Matrizes

- As matrizes são variáveis compostas multidimensionais:
 - São equivalentes aos vetores, mas com mais dimensões.
 - Acesso também é feito por índice que varia em cada uma das dimensões (por exemplo, linha e coluna).
- Podem ser definidas como sendo um vetor, onde cada elemento é um outro vetor.
- Usada para representar tabelas, grades e relações.

Exemplo ilustrativo

- Em uma turma de alunos, cada aluno possui 5 (cinco) notas.
- A turma possui 40 alunos. Portanto, é necessário organizar as notas em linhas e colunas, da seguinte forma:

	N1	N2	N3	N4	N5
Aluno 1	5.0	4.5	8.0	9.5	7.0
Aluno 2	4.0	4.5	3.5	7.5	2.0
Aluno 3	5.0	8.5	9.0	9.0	7.5
...					
Aluno 40	0.0	5.5	6.0	6.5	8.0

- Linhas → alunos
- Colunas → notas

Exemplo ilustrativo

- Em uma turma de alunos, cada aluno possui 5 (cinco) notas.
- A turma possui 40 alunos. Portanto, é necessário organizar as notas em linhas e colunas, da seguinte forma:

	0	1	2	3	4
0	5.0	4.5	8.0	9.5	7.0
1	4.0	4.5	3.5	7.5	2.0
2	5.0	8.5	9.0	9.0	7.5
...					
39	0.0	5.5	6.0	6.5	8.0

- Linhas → alunos
- Colunas → notas

Exemplo ilustrativo

- Em uma turma de alunos, cada aluno possui 5 (cinco) notas.
- A turma possui 40 alunos. Portanto, é necessário organizar as notas em linhas e colunas, da seguinte forma:

```
turma = [ [5.0, 4.5, 8.0, 9.5, 7.0] ,  
          [4.0, 4.5, 3.5, 7.5, 2.0] ,  
          [5.0, 8.5, 9.0, 9.0, 7.5] ,
```

...

- Linhas → alunos
 - Colunas → notas
- ```
[0.0, 5.5, 6.0, 6.5, 8.0]]
```

# Acesso aos elementos de uma matriz

- Também feito por índices que percorrem cada uma das dimensões (como se fossem coordenadas).
- Sintaxe: `matriz [linha] [coluna]`
- Exemplos:

```
turma[0][1] = 4.5
turma[2][4] = 7.5
turma[21][1] = ??
turma[40][4] = erro de índice
```

|     | 0   | 1   | 2   | 3   | 4   |
|-----|-----|-----|-----|-----|-----|
| 0   | 5.0 | 4.5 | 8.0 | 9.5 | 7.0 |
| 1   | 4.0 | 4.5 | 3.5 | 7.5 | 2.0 |
| 2   | 5.0 | 8.5 | 9.0 | 9.0 | 7.5 |
| ... |     |     |     |     |     |
| 39  | 0.0 | 5.5 | 6.0 | 6.5 | 8.0 |

# Percorrendo os elementos de uma matriz

- Usando dois índices e laços aninhados:
- Exemplo:

```
for i in range(len(turma)) :
 for j in range(len(turma[i])) :
 print(turma[i][j])
```

|     | 0   | 1   | 2   | 3   | 4   |
|-----|-----|-----|-----|-----|-----|
| 0   | 5.0 | 4.5 | 8.0 | 9.5 | 7.0 |
| 1   | 4.0 | 4.5 | 3.5 | 7.5 | 2.0 |
| 2   | 5.0 | 8.5 | 9.0 | 9.0 | 7.5 |
| ... |     |     |     |     |     |
| 39  | 0.0 | 5.5 | 6.0 | 6.5 | 8.0 |

# Criando uma matriz "zerada"

- Programa que cria uma matriz **n x m** com zeros em todas as suas posições:

```
n = int(input('Digite o número de linhas n da matriz: '))
m = int(input('Digite o número de colunas m da matriz: '))
matriz = []
for i in range(n):
 linha = []
 for j in range(m):
 linha.append(0) # acrescenta zeros em todas as linhas
 matriz.append(linha)
print(matriz)
```

# Criando uma matriz com valores lidos

- Programa que cria uma matriz **n x m** com zeros em todas as suas posições:

```
n = int(input('Digite o número de linhas n da matriz: '))
m = int(input('Digite o número de colunas m da matriz: '))
matriz = []
for i in range(n):
 linha = []
 for j in range(m):
 linha.append(int(input())) # valores lidos do teclado
 matriz.append(linha)
print(matriz)
```

# Como imprimir no formato de tabela

- Programa que imprime os elementos de uma matriz no formato de uma tabela (linha por linha):

```
for i in range(n):
 print(matriz[i])
```

**i = 0** - imprime a primeira linha e pula uma linha  
**i = 1** - imprime a segunda linha e pula uma linha  
**i = 2** - imprime a terceira linha e pula uma linha  
...  
**i = (n-1)** - imprime a última linha e pula uma linha

# Matrizes multidimensionais

- Matrizes podem ter mais de duas dimensões.
- Exemplo: turmas  $\rightarrow$  alunos  $\rightarrow$  notas.
- Nesses casos, o acesso é feito com múltiplos índices, sendo necessário um índice para percorrer cada uma das dimensões da matriz.

# Desafios para pensar

- Escreva um programa que soma todos os elementos de uma matriz **N x M**, cujos valores são lidos do teclado.
- Escreva um programa que calcula a média dos elementos de uma matriz **N x M**, cujos valores são lidos do teclado.
- Escreva um programa que conta quantos valores pares e ímpares existem em uma matriz **N x M**, cujos valores são lidos do teclado.
- Escreva um programa que busca o maior e o menor elemento de uma matriz **N x M**, cujos valores são lidos do teclado.

# Resumo da aula

- Estudamos vetores: tipo composto unidimensional.
  - Armazena múltiplos valores em uma única variável.
  - Acesso por índice.
- Estudamos matrizes: tipo composto multidimensional.
  - Organizam os dados em linhas e colunas.
  - Podem ter mais de duas dimensões (dependendo da aplicação).
  - Acesso por índice.
- Índices de acesso sempre começam em zero
- Uso combinado de laço `for`, `len()` e `listas` em Python.

# Dicas e próximos passos

- Implemente e teste os códigos apresentados nessa aula em IDEs e interfaces interativas:
  - IDEs: Thonny, PyCharm, VSCode;
  - Interfaces interativas: IDLE, Jupyter Notebook, Google Colab.
- Aprender algoritmos exige empenho, dedicação e muitos .... muitos exercícios;
- Leia os materiais indicados como referências;
- Busque outras fontes;
- Faça pesquisas e exercícios;
- Comprometa-se com o curso e com a disciplina.

# Referências

BANIN, S. L. **Python 3: Conceitos e Aplicações - Uma Abordagem Didática**. São Paulo: Erica, 2018. ISBN 9788536530253. **Disponível na Biblioteca Digital da UFMS.**

MANZANO, J. A. N. G. **Algoritmos: Lógica para Desenvolvimento de Programação de Computadores**. 29 ed. São Paulo: Erica, 2019. ISBN 9788536531472. **Disponível na Biblioteca Digital da UFMS.**

PERKOVIC, L. **Introdução à computação usando Python: um foco no desenvolvimento de aplicações**. Rio de Janeiro: LTC, 2016. ISBN 9788521630937. **Disponível na Biblioteca Digital da UFMS.**

WENTWORTH, P.; ELKNER, J.; DOWNEY, A. B.; MEYERS, C. **How to think like a computer scientist: Learning With Python 3**. [S.L.: S. N.], 2012. Disponível em: <https://link.ufms.br/Vow0N>. Acesso em 05 mai. 2025.

# Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

